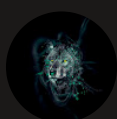


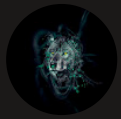


Query Data with DynamoDB



Kiprono Yegon

```
}
nextworksampladata $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"202"}}' \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "Item": {
    "ContentType": {
      "S": "Video"
    },
    "Title": {
      "S": "Don't miss out!"
    }
  },
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 0.5
  }
}
nextworksampladata $ █
```



Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a fully managed NoSQL database service that stores and retrieves large amounts of data quickly and efficiently it is important because of its speed and flexibility

How I used Amazon DynamoDB in this project

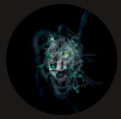
In today's project, I used Amazon DynamoDB to create tables and load data into the attributes in the tables. I also did a transaction.

One thing I didn't expect in this project was...

One thing I didn't expect in this project is the ease of using the AWS CLI to carry out tasks in the DynamoDB

This project took me...

This project took me about an hour



Querying DynamoDB Tables

A partition key is the filter that DynamoDB will use to split up and find data. Partition key values don't have to be unique.

A sort key is a secondary key used to filter your query results again! Sort keys work after the partition key i.e. you still have to use the partition key to split up your data first, and then the sort key partitions your data again.

Comment Autosave [View table details](#)

▼ Scan or query items [Info](#)

☐ Scan ☒ Query

Select a table or index: **Table - Comment** Select attribute projection: **All attributes**

Partition key

Attribute: **Id** Value: **I have a question/Just Complete Project #7 Dependencies and CodeArtifacts**

Sort key - optional

Attribute: **CommentDateTime** Value: **Greater than** **2024-09-01** ☐ Sort descending

► Filters - optional

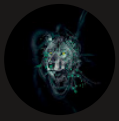
[Run](#) [Reset](#)

Completed - Items returned: 1 - Items scanned: 1 - Efficiency: 100% - RCUs consumed: 0.5

Table: Comment - Items returned (1) [Actions](#) [Create Item](#)

Query started on May 21, 2026, 21:24:45

☐	Id (String)	CommentDateTime (String)	Message	PostedBy
☐	I have a question/Just...	2024-09-01T19:58:22.947Z	Legendary	User Abhishek



Limits of Using DynamoDB

I ran into an error when I queried for the data PostedBy user Abdulrahman. This was because it did not have a partition key.

Insights we could extract from our Comment table includes the comments posted by a user in a given range of time. Insights we can't easily extract from the Comment table includes the posts a user posted.

▼ Scan or query items [Info](#)

☐ Scan ☒ Query

Select a table or index: Table - Comment ▼ Select attribute projection: All attributes

Partition key

Attribute: Id Value: Enter attribute value

ⓘ The partition key filter cannot be empty.

Sort key - optional

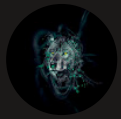
Attribute: CommentDateTime Value: Greater than ▼ Enter attribute value ☐ Sort descending

▼ Filters - optional

Attribute name	Condition	Type	Value	
Q. PostedBy	Equal to	String	User Abdulrahman	Remove

[Add filter](#)

[Run](#) [Reset](#)

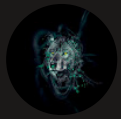


Running Queries with CLI

A query I ran in CloudShell was to find the consistent reads. This query will return consistent reads

Query options I could add to my query are the table name, key, project expression and the consumed capacity.

```
}
nextworksampladata $ aws dynamodb get-item \
> --table-name ContentCatalog \
> --key '{"Id":{"N":"282"}}' \
> --projection-expression "Title, ContentType, Services" \
> --return-consumed-capacity TOTAL
{
  "Item": {
    "ContentType": {
      "S": "Video"
    },
    "Title": {
      "S": "Don't miss out!"
    }
  },
  "ConsumedCapacity": {
    "TableName": "ContentCatalog",
    "CapacityUnits": 0.5
  }
}
nextworksampladata $ █
```



Transactions

A transaction is a group of operations that all have to succeed - if any of the operations in the group fails, none of the changes get applied. This makes sure that any change to your database is consistent across all your tables!

I ran a transaction using the AWS CLI. This transaction did two things, it updated the Forum and and comments section

```
nextworksampledatab $ ls
Comment.json ContentCatalog.json Forum.json Post.json
nextworksampledatab $ aws dynamodb transact-write-items --client-request-token TRANSACTION1 --transact-items '[
>   {
>     "Put": {
>       "TableName" : "Comment",
>       "Item" : {
>         "Id" : {"S": "Events/Do a Project Together - NextWork Study Session"},
>         "CommentDateTime" : {"S": "2024-9-27T17:47:30Z"},
>         "Comment" : {"S": "Excited to attend!"},
>         "PostedBy" : {"S": "User Connor"}
>       }
>     },
>     {
>       "Update": {
>         "TableName" : "Forum",
>         "Key" : {"Name" : {"S": "Events"}},
>         "UpdateExpression": "ADD Comments :inc",
>         "ExpressionAttributeValues" : { ":inc": {"N" : "1"} }
>       }
>     }
>   ]'
nextworksampledatab $ █
```



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

